PROXMOX AUTOMATION SERIES · PHASE 4

The Request

Form

One typed form on Semaphore UI: profile, hostname, IP, VLAN, and a custom disk list. Behind it, Ansible-native provisioning replaces Terraform, serial-tagged disks and the full agent layer build themselves, and a delivery gate signs the handover receipt.

PVE 9.2 · SEMAPHORE UI 2.19.11 · ANSIBLE-CORE 2.21
COMMUNITY.PROXMOX 2.0.0 · SQLITE DIALECT · SURVEY FORM
AUGUST 2026 · COMPANION REPO: PROXMOX-AUTOMATION

FILL · RUN · DELIVER

Contents

1. What Phase 4 Adds

This is the fifth module of the Proxmox Automation Series, and the one the whole series was pointed at: the request form. Phases 1 through 3 built the pieces, one closed phase at a time, and each ended with its claims field-verified. A template exists on NFS. Clones come up with cloud-init networking. Serial-tagged disks converge into mounted XFS. Agents install, verify themselves, and forward logs in the QRadar pattern. What remained was the part the engineers actually touch twenty to ninety times a day: the intake. A request arrives, someone types specifications into a form, and a fully configured VM comes out the other end with a receipt.

The original Phase 4 plan, as recorded in the series notes, was a custom web application: an Angular frontend, a FastAPI backend, a job engine, a live timeline. That plan was abandoned during planning week, deliberately, and the reasons are worth more than the plan was. Two findings did the abandoning. The first came from the ready-made UI evaluation: the wheel already existed, and the leading candidate was healthier than expected. The second came from the Ansible collection research, and it quietly retired Terraform from the provisioning path. Both findings were verified against primary sources on 28 August 2026, not read off a blog.

Finding one: AWX is frozen, Semaphore is not, and the form exists

The ready-made-UI shortlist had two names on it. AWX, the Red Hat sponsored upstream of the commercial Ansible Automation Platform, is the famous one: 15.5 thousand stars, a real web UI, workflow builder, RBAC, surveys. Semaphore UI is the smaller one: a single Go binary, a Vue frontend, task templates, environment variables, and, as of this year, typed survey variables. The GitHub release API told the story that decides it:

Question	AWX (ansible/awx)	Semaphore (semaphoreui/semaphore)
Last stable release	24.6.1, published 2024-07-02; no release since	v2.19.11, published 2026-08-27; releases every 1-2 weeks
Operator status	"releases paused during a large scale refactoring" (project statement, Sep 2025); Red Hat contribution withdrawn	Active daily development, 14k stars, MIT license
Install footprint	Kubernetes (k3s) plus operator; roughly a 4 GB VM for the lab	One Go binary plus systemd; fits the existing LXC pattern
Request form	Surveys: strings and a few types; no dynamic disk list either	Survey variables: string, integer, enum, multiline text, secret
Audit trail	Strong (job history, credentials, RBAC)	Strong for this use (every task stores who, when, which form values, full output)

Table 1. The evaluation that redirected Phase 4, verified from the GitHub API and each project's release notes on 28 Aug 2026.

The honest summary: AWX would have worked technically. Every requirement maps to an AWX feature, mostly without workarounds. But binding a twenty-to-ninety-deploys-a-day operations pipeline to a release train that has not moved in two years is the kind of decision that looks free today and costs a weekend next year. Semaphore is not a compromise here; its survey feature covers every field the request form needs (chapter 5), and its release cadence is the opposite of frozen. The custom-UI plan, meanwhile, failed on ownership arithmetic: a bespoke web app is another system to build, test, and live with, and it duplicates what Semaphore already ships.

> **0**
>
> a custom web UI at all. The decision inverted the series' usual build-donot-buy reflex because the thing being built was not differentiating: the value lives in the playbooks, and the playbooks run under any runner.

Finding two: the Proxmox collection grew up, and Terraform retired

The second finding came from checking whether the provisioning path could drop its second language. In the community.general era, the answer was no: the proxmox modules there could clone, but disk management was too thin for the Phase 2 serial design. The proxmox content has since split into its own collection, community.proxmox, and its 2.0.0 line (verified from the Galaxy API and the module source itself) closes the gap. The proxmox_disk module takes a serial parameter natively, up to 20 bytes, which is exactly the PVE limit; proxmox_kvm clones templates and applies full specifications; proxmox_vm_info answers existence queries. The entire Phase 1-to-3 Terraform output can now be

produced by Ansible alone.

That retires Terraform from the provisioning path, and the retirement is about state, not fashion. The lab's VMs are created by the pipeline and deleted by humans, outside it. Every terraform plan after such a deletion shows drift, and every import-or-taint dance is operator time spent reconciling a state file with reality. The Ansible-native path has no state to reconcile: a VM either exists (looked up by name) or it does not, and the playbook handles both cases explicitly. The terraform/ directory stays in the repository, runnable for reference and for the template build, but the deploy flow no longer touches it.

Phase 4 object	Replaces	Notes
`00_provision_vm.yml (Ansible-native)`	terraform apply of vm.tf	clone, specs, cloud-init, serial disks, start; no state file
`Semaphore on sem-01 (CT 901)`	the planned Angular + FastAPI app	typed survey form, task audit, zero custom code to own
`07_delivery_check.yml`	the manual handover checklist	mounts, services, forwarding, tagged message; green output is the record
`profiles.yml (db-standard, web-small, app-medium, custom)`	per-request tfvars editing	the preset layer of the form; serials derived from roles
`semaphore/ (wiring reference + env template)`	nothing; new	the UI objects, the survey design, the security notes

Table 2. What shipped, and what each piece replaces.

What was verified before this document was written

The series rule is that claims get verified, and Phase 4 had an unusual amount of verifying available because the artifacts themselves could run in a sandbox. The Semaphore v2.19.11 binary was downloaded, configured, migrated, given an admin user, and started; it answered HTTP 200. The one path that validation never exercised was the login itself, and the field collected that debt within minutes of the first sign-in: the request answered HTTP 500 until the config carried the session cookie keys (chapter 3 and chapter 8 record the gap and the fix; the sandbox now tests the login endpoint too). The community.proxmox 2.0.0 tarball was pulled from Galaxy and its module source read line by line for the behaviors the playbook depends on. Every playbook passed syntax checking under ansible-core 2.21, and the profile resolution logic ran as four standalone logic tests (custom, db-standard, web-small, app-medium), which caught two real bugs before they could reach the lab. Both bugs are documented in chapter 8 with their fixes.

2. The Engine: Ansible-Native Provisioning

The heart of Phase 4 is one new playbook, 00_provision_vm.yml, and the module behaviors it is built around. This chapter walks the playbook step by step, then records the three community.proxmox behaviors that shaped it. Those behaviors are not documented prominently anywhere; they were found by reading the collection source, and they are the difference between a playbook that works and one that silently does half its job.

The six steps

The play runs on localhost with connection: local; it speaks to Proxmox over HTTPS, never SSH. Six steps, in order:

- Validate the request. Profile name, DNS-safe hostname, dotted IPv4, VLAN range; for custom profiles also the vCPU and RAM enum sets and the disk JSON. The form is UX; this assert is the gate.
- Resolve the profile into a specification: vCPU, RAM, and the disk list with serial, VG, LV, and mount derived from each role.
- Look up existing VMs with that name (proxmox_vm_info). Found means reconfigure; not found means clone. This lookup is the idempotency guard.
- Clone the golden template (full copy on nfs-shared), then apply the specification in a second task: CPU, RAM, net0 with optional VLAN tag, the cloud-init drive, ipconfig0, SSH keys, DNS, agent, onboot, tags.
- Attach the serial-tagged data disks on scsi1..n (proxmox_disk), one per role, with create: regular so re-runs are no-ops on existing disks.
- Start the VM, wait for SSH, and add_host it into the provisioned group so plays 01 through 07 pick it up in the same run.

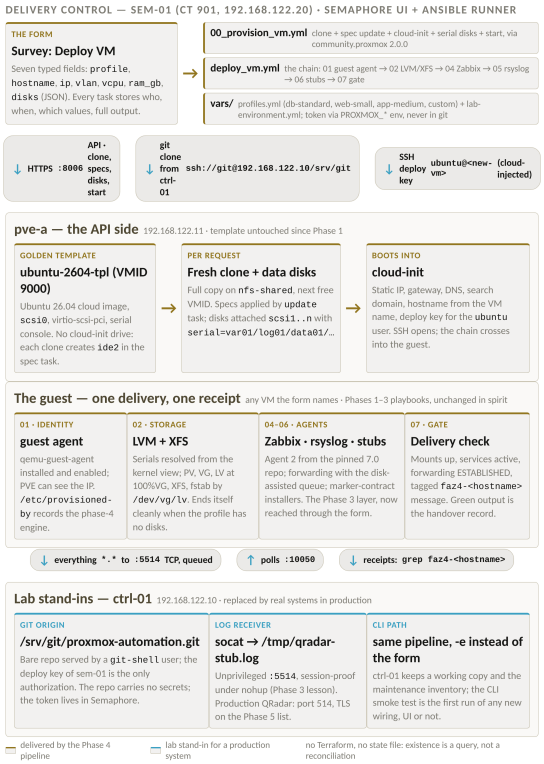

Figure 1. The Phase 4 delivery path: one form, one pipeline, one receipt. sem-01 runs Semaphore and the playbooks; ctrl-01 hosts the git origin and the log receiver stub; pve-a serves the API.

Three module behaviors that shaped the playbook

The first behavior is the one that would have been invisible until the first production run. In proxmox_kvm's clone branch, the module forwards only six parameters to the clone API call: format, full, pool, snapname, storage, and target. Everything else passed alongside clone, the cores, the memory, the network, the cloud-init settings, is silently dropped. A single-task "clone with specs" that looks correct in every blog post produces a clone with the template's defaults. The playbook therefore clones first, then applies the specification in a second task with update: true against the new VMID.

The second behavior lives in that update task. The module's update path deliberately skips disks and network interfaces unless update_unsafe: true is set; the safety rail exists because rewriting net0 on a production VM is risky. A fresh clone has nothing to clobber, and the specification explicitly includes net0 and ide2, so the playbook sets the flag. On re-runs against an existing VM the values are identical, and the configuration call is a no-op.

The third behavior is idempotency, or the lack of it. In clone mode the module does not check whether a VM with the target name already exists; it allocates the next free VMID and clones again. A re-run of a naive playbook therefore produces a second VM with the same name and a different ID. The playbook level guard in step 3 is not an optimization; it is what makes the whole chain re-runnable. The explicit-VMID variant has its own semantics: passing a newid that already exists exits with changed: false

and that VMID, which the playbook treats as an existing VM and reconfigures.

Behavior	Source evidence	How the playbook answers
Clone drops spec parameters	create_vm() clone branch: only format, full, pool, snapname, storage, target reach the API	Two tasks: clone, then update: true with the full spec
Update skips net and ide	update path strips net, scsi, virtio, ide and more unless update_unsafe: true	update_unsafe: true on a fresh clone; identical values on re-runs
Clone mode is not idempotent by name	newid defaults to next-free; no name check in the clone path	proxmox_vm_info by name first; found means reconfigure, not clone
Cloud-init drive must exist before its parameters	the Phase 1 template never had one (initialization lived on the workload)	ide2: nfs-shared:cloudinit created in the same update task

Table 3. Module behaviors from the community.proxmox 2.0.0 source, and the playbook's answer to each.

Profiles: the preset layer

The request form does not ask for disk serials, VG names, or mount points, and it never will. It asks for a profile, and the profile resolves to a disk list whose entries carry only a role and a size. Everything else is derived: role data becomes serial data01, volume group vg_data, logical volume lv_data, mount point /srv/data. The derivation is deliberately identical to the field-proven Phase 2 layout, so the db-standard profile produces exactly the disk set the lab already ran with, and the unchanged 02_storage.yml builds either path's disks.

Profile	vCPU / RAM	Disks (role -> mount)
db-standard	8 / 32 GB	var -> /srv/var, log -> /srv/log, data -> /srv/data, backup -> /srv/backup
web-small	2 / 4 GB	none (the storage play ends itself cleanly)
app-medium	4 / 16 GB	app -> /srv/app
custom	from the form enums	from the form's JSON; roles limited to var, log, data, backup, app, scratch

Table 4. The profile set. Sizes are production-shaped; lab validation should use custom with small sizes.

The chain: deploy_vm.yml

Semaphore runs exactly one playbook per task template, so the chain lives in deploy_vm.yml: seven import_playbook lines in fixed order, 00 through 07. One deliberate omission: 03_storage_verify.yml is

not in the chain, because it reboots the VM to prove boot persistence, and a reboot in the middle of every deploy would double delivery time for no daily value. It remains available as its own template for on-demand proof, exactly as in Phase 2. The chain also documents the two-inventory split: the deploy flow uses provisioning.yml, whose provisioned group is empty on purpose and fills at runtime via add_host; the maintenance inventory hosts.yml keeps the known VMs, so a deploy can never accidentally touch them. A root-level ansible.cfg mirrors the ansible one and exists for how Semaphore invokes the runner: it clones the repository and executes from the clone root.

0

a second orchestration language. Everything Terraform did for this pipeline is now done by the same tool that configures the guest, against the same API, with no state file to reconcile.

3. sem-01: the Semaphore Container

Semaphore lives in its own unprivileged LXC container, CT 901, created by scripts/create-sem-01.sh on pve-a and configured by scripts/sem-01-setup.sh inside it. The sizing question has a boring answer: Semaphore is one Go binary with an embedded SQLite database, and the Ansible runs happen in the same container; 2 vCPU and 2 GB run the lab comfortably, and the script bumps nothing because nothing in the lab needs it.

The scripts encode the install decisions so the container can be rebuilt in minutes, and every non-obvious decision in them was verified against the artifacts rather than a blog. The .deb from the GitHub release was downloaded and inspected: it contains exactly one file, /usr/bin/semaphore, so the setup script creates the systemd unit itself, from the unit template in the official "Run as a service" documentation. The binary was then run for real: migrations applied, an admin user created with the users add subcommand, the server started and answered HTTP 200. Two findings from that session are now baked into the script and documented here because they will otherwise cost someone an evening.

The port must be a string

The first run of semaphore migrate against a hand-written config.json failed with a Go panic: json: cannot unmarshal number into Go struct field ConfigType.port of type string. The port field must be quoted, "port": "3000", not 3000. The panic is loud, the message names the field, and every example config in the wild seems to have the string form, but a JSON author writing it fresh from first principles will write a number. The setup script writes the config with the quotes; the error and its fix are in the

troubleshooting matrix for whoever ignores the script.

The cookie keys must exist, or nobody logs in

The fourth field finding arrived at the login form, and it is the subtlest of the four: v2.19 signs its session cookies, and the binary requires two secrets in the config, cookie_hash and cookie_encryption. Without them the server starts, the form renders, the password verifies, and the sign-in still dies with HTTP 500 and "Failed to create session", because the cookie cannot be encoded. Two details make it worse than it sounds. The field names circulating in older guides, cookie_hash_key and cookie_encryption_key, belong to earlier versions and are silently ignored now; the working names above were read out of the binary itself. And `semaphore setup`, the interactive wizard, generates both keys - so everyone who follows the official path never sees the failure, and everyone who writes a minimal config by hand meets it at the worst possible moment, one field before the dashboard. The setup script generates both keys with od -N (the pipefail-safe bounded read; head -c caused bug one) and writes them into the config; the sandbox validation now proves the login round trip, 204 plus a Set-Cookie header and an authenticated follow-up call, not just the HTTP 200 of the front page.

SQLite is the default, and that is fine

The current Semaphore configuration documentation lists the dialect values as sqlite (the default), postgres, and mysql. For a single-container lab the default is the right call: no database server to install, patch, or back up separately, and the database file lives in /var/lib/semaphore where a pct snapshot captures it. The switch to PostgreSQL is a set of environment variables, SEMAPHORE_DB_DIALECT and friends, so the upgrade path exists if remote runners or high availability ever arrive; the guide records that path and deliberately does not walk it.

SEM-01: /ETC/SEMAPHORE/CONFIG.JSON (WRITTEN BY SEM-01-SETUP.SH)

```
{
"dialect": "sqlite",
"sqlite": { "host": "/var/lib/semaphore/semaphore.db" },
"port": "3000",
"tmp_path": "/var/lib/semaphore/tmp",
"demo": false,
"cookie_hash": "<64 hex chars, generated at install>",
"cookie_encryption": "<32 hex chars, generated at install>"
}
```

The service runs as a dedicated semaphore user, never root, with WorkingDirectory set to the data directory. The setup script generates the admin password when none is supplied, prints it once, and expects it changed on first login. It also generates the deploy keypair at /home/semaphore/.ssh/id_ed25519; that keypair is the pipeline's identity, one key doing two jobs in the lab: it authenticates sem-01 to the git origin on ctrl-01, and its public half is injected into every new VM by cloud-init so the playbooks can SSH in. The production-grade split, a separate key for each job, is a

five minute change documented in semaphore/README.md.

The Ansible environment inside the container

sem-01 doubles as the Ansible control node for the deploy flow, so it needs the toolchain. Ubuntu 26.04 blocks pip system-wide installs under PEP 668, the same externally-managed-python wall this series met in Phase 1, and the clean answer here is apt: ansible, python3-proxmoxer, python3-requests, python3-jmespath, and python3-netaddr all come from the distribution repository. The collections install as the semaphore user into its home with ansible-galaxy: community.proxmox, community.general, and ansible.posix, exactly the set requirements.yml declares. If python3-proxmoxer is ever absent from the repository, the fallback is a venv under the service user; the setup script notes the path.

> **0**
>
> a database server in the container. SQLite is the project's own default dialect; the file sits next to the job data in /var/lib/semaphore, and one pct snapshot captures the whole service state.

4. The Git Origin on ctrl-01

Semaphore needs a git repository URL; it clones the repository for every task run, and that is the whole point of this chapter: Semaphore runs on sem-01, and the only part of ctrl-01 it can reach is the network. The working copy that lived well enough as a plain directory through Phases 1-3 now needs a servable home - a bare repo behind a dedicated git user whose shell is git-shell.

CTRL-01 (ROOT): HOST THE REPOSITORY FOR SEMAPHORE

```
useradd -m -s /usr/bin/git-shell git
mkdir -p /srv/git
git init --bare --initial-branch=main /srv/git/proxmox-automation.git

# authorize sem-01's deploy key (sem-01-setup.sh step 7 output):
install -d -m 700 -o git -g git /home/git/.ssh
DEPLOY_KEY="ssh-ed25519 AAAAC3NzaC1lZDI1NTE5\
AAAAICaUb7zvNq8LGILOM96t9LeOBVjLjXuHnrydb+0k6fdU semaphore@sem-01"
echo "$DEPLOY_KEY" > /home/git/.ssh/authorized_keys
chown git:git /home/git/.ssh/authorized_keys
chmod 600 /home/git/.ssh/authorized_keys

# refresh the working copy from the CURRENT zip first (the drift
# trap is documented below), then publish:
cd /root/proxmox-automation
git init -b main && git add -A
git commit -m "phase 4: delivery pipeline"
git push /srv/git/proxmox-automation.git main
chown -R git:git /srv/git/proxmox-automation.git
git config --global --add safe.directory /srv/git/proxmox-automation.git
```

The chapter earned two errata rows in Table 8 the first time it ran for real, on 28 August 2026. Plain git init still creates a branch called master - the default changes only in Git 3.0 - while the bare repo, the push command, and the Semaphore repository object all say main, so the original block pushed a branch that did not exist and git answered "error: src refspec main does not match any"; the corrected block above creates the branch with -b main from the start. The working copy itself had drifted too: the copy on ctrl-01 still held the Phase 3 tree, thirty files with no deploy_vm.yml and no provisioning.yml, and a push in that state would have handed Semaphore a repository without the playbook it exists to run. Refresh the working copy from the current zip before the first push, and read the commit list as a receipt: if deploy_vm.yml is not in what git add -A picked up, nothing downstream of the push can work.

The two lines after the push settle a boundary git has cared about since CVE-2022-24765: the repository was created by root, but the process that serves it over SSH runs as the git user, and git refuses to operate on a repository owned by somebody else - the failure mode is a clone that answers "fatal: detected dubious ownership". Handing the directory to the git user fixes the serving side; the safe.directory entry keeps root's local-path push working afterwards, because from that moment the repository is no longer root's either.

PVE-A (ROOT): PROVE THE CHAIN BEFORE THE UI

```
pct exec 901 -- bash -c "ssh-keyscan -H 192.168.122.10 \
>> /home/semaphore/.ssh/known_hosts"
pct exec 901 -- chown semaphore:semaphore /home/semaphore/.ssh/known_hosts
pct exec 901 -- chmod 600 /home/semaphore/.ssh/known_hosts
pct exec 901 -- sudo -u semaphore git ls-remote \
ssh://git@192.168.122.10/srv/git/proxmox-automation.git
```

The ls-remote at the end is the chapter's receipt: two lines, the same commit hash twice, for HEAD and for refs/heads/main. That one hash proves the entire chain - the deploy key, the authorized_keys file that holds it, git-shell, the bare repo, and its ownership - so chapter 5 can only fail on its own settings. The keyscan above it exists because the semaphore user has never spoken to ctrl-01 before, and a non-interactive clone cannot answer an interactive host-key question. One prerequisite both commands assume: sshd listening on ctrl-01 - ss -tln on ctrl-01 is the ten-second check, apt-get install -y openssh-server the fix if the container lacks it.

The git-shell shell is deliberate: the git user can serve repositories and nothing else, no interactive login, no command execution beyond the git protocol. In Semaphore the repository URL is ssh://git@192.168.122.10/srv/git/proxmox-automation.git with the deploy key selected as the access key. The push from the working copy is local, straight to the bare path, no SSH involved; only sem-01's clones go over the network. When the repository outgrows the lab, the same Semaphore object points at any hosted git URL and nothing else changes.

One habit worth keeping from day one: the working copy on ctrl-01 stays the editing surface, and pushes go to the origin before Semaphore runs them. The tokenless clone URL means the repository carries no secrets; the Proxmox API token lives in Semaphore's environment, the deploy key lives in Semaphore's key store and the git user's authorized_keys, and the git history stays clean of credentials. The one token the lab created in Phase 1 has already been pasted into a chat window once; the series owes it no second exposure.

5. Wiring the UI: Objects and the Form

Everything in Semaphore hangs off one project, and the project needs five objects before the first task can run. Each is created once in the web UI; semaphore/README.md in the repository keeps the same reference for the day the UI needs rebuilding. None of the five contains a secret that git would mind holding, except the one that deliberately holds only secrets and lives in Semaphore, never in the repository.

Object	Name	Content
Key Store entry	deploy-key	the private half of sem-01's id_ed25519; used for repository access
Repository	proxmox-automation	ssh://git@192.168.122.10/srv/git/proxmox-automation.git, branch main, access key deploy-key
Inventory	provisioning	the content of ansible/inventory/provisioning.yml: localhost plus an empty provisioned group
Environment	proxmox-lab	the four PROXMOX_* variables; the API token lives here and only here
Task Template	Deploy VM	Ansible type, playbook ansible/playbooks/deploy_vm.yml, the inventory and environment above, and the survey

Table 5. The five Semaphore objects behind the form.

The environment deserves one paragraph of why. The playbook reads the Proxmox connection through lookup(env, ...) for exactly four names: PROXMOX_HOST, PROXMOX_USER, PROXMOX_TOKEN_ID, PROXMOX_TOKEN_SECRET. The community.proxmox modules document those environment names themselves, which means the pipeline and the collection agreed on the convention before they met. On the CLI the same variables come from a shell export; in Semaphore they come from the environment object attached to the template. Either way the token never appears in the repository, in a playbook, or in a task log, and a rotated token is a one-field edit in the UI.

The form: survey variables on the Deploy VM template

Field	Type	Values or example	Notes
`vm_profile`	enum	db-standard, web-small, app-medium, custom	the preset layer; most requests stop here
`vm_hostname`	string	vm-app-01	lowercase DNS-safe; doubles as the PVE VM name
`vm_ip`	string	192.168.122.60	dotted IPv4, no CIDR
`vm_vlan`	integer	0	0 means untagged
`vm_vcpu`	enum	2, 4, 8, 16, 32	custom profile only
`vm_ram_gb`	enum	4, 8, 16, 32, 64, 128	custom profile only
`vm_disks`	text (multiline)	JSON list, below	custom disks; overrides the profile list

Table 6. The seven survey fields. Enums where the pipeline has opinions; free text only where the request genuinely varies.

Two of these choices carry the whole design. The vCPU and RAM fields are enums, not number inputs, because the form should only ever offer what the pipeline allows: "the customer wants seven vCPUs" is a conversation, not a form value. And the disk list is the only free-form field, because custom storage requests genuinely vary; it accepts a JSON list of role and size pairs, and everything a mistyped disk spec would otherwise get wrong is derived instead of typed. The serials, the volume groups, the logical volumes, and the mount points are computed from the role names. Nobody can typo "data01" into a field that does not exist.

THE VM_DISKS JSON (CUSTOM REQUESTS ONLY)

```
[
{"role": "data", "size_gb": 1024},
{"role": "log", "size_gb": 50},
{"role": "backup", "size_gb": 500}
]
```

The assert block in 00_provision_vm.yml re-validates every field server-side, because the survey is only the UI path: extra-vars on the CLI and the task API both bypass it. The allowed roles are a closed list, sizes must be positive integers, roles must be unique, and the hostname must match a DNS-safe pattern. A request that fails validation fails in seconds with a message that names the field, long before any clone exists to clean up.

0

webhooks and schedules for the first delivery. Both exist in Semaphore, both can pre-fill survey values, and a customer self-service portal behind a webhook is a legitimate Phase 5 candidate once the approval workflow question has an answer.

6. First Deploy, Two Ways

The pipeline can prove itself twice: once on the CLI, where the failure modes are plain, and once through the form, which is the point of the phase. The CLI run is recommended first, not as ceremony but because a broken UI wiring and a broken playbook look identical in a task log, and the CLI removes the UI from the variables.

The CLI smoke test

```
CTRL-01 (OR SEM-01): ONE CUSTOM VM, SMALL DISKS, NO UI INVOLVED

export PROXMOX_HOST=192.168.122.11 \
PROXMOX_USER=terraform@pve \
PROXMOX_TOKEN_ID=provider \
PROXMOX_TOKEN_SECRET=<the token UUID>

cd ansible
ansible-playbook -i inventory/provisioning.yml playbooks/deploy_vm.yml \
-e vm_profile=custom -e vm_hostname=vm-faz4-01 -e vm_ip=192.168.122.60 \
-e vm_vcpu=2 -e vm_ram_gb=4 \
-e 'vm_disks=[{"role":"data","size_gb":10}]'

# the run ends with the 07 delivery receipt; the receiver-side proof:
# grep faz4-vm-faz4-01 /tmp/qradar-stub.log (on ctrl-01)
```

The playbook chain shows its shape in the recap: the localhost plays run first, the clone task reports the new VMID, the disk loop labels itself scsi1 data01 10G, and then the story crosses into the guest: guest agent, storage, Zabbix, rsyslog, the stub installers, and the delivery gate. A green run prints the receipt line that ends with the receiver-side instruction, and the grep on ctrl-01 is the field receipt: the faz4 tag, the hostname, the timestamp. The tag pattern continues the series convention from Phase 3; a delivery check message is distinguishable from noise in the stub log for exactly as long as someone needs to find it.

Through the form

The second run is the demonstration. In Semaphore: New Task on the Deploy VM template, the survey appears, fill the seven fields, Run. The task page shows the run live; Semaphore streams playbook output as it happens, and the run ends on the same delivery receipt. The task record now holds the complete audit entry the manual process never had: who launched it, when, every form value, the full output, and the duration. That record is the answer to a question the voice memos asked without asking: when something ships wrong, the first question is not "who did what" but "which parameters did the machine actually receive", and the answer is one click deep.

Re-runs are safe by construction, and worth proving once: run the same task again with the same hostname. The existence guard finds the VM, the clone is skipped, the specification is re-applied as a no-op, the disks are already attached, the VM is already running, and the guest plays converge with changed counts at zero. One caution travels with the claim: re-running with a DIFFERENT profile against the same hostname re-applies CPU, memory, and tags to the existing VM, and disk changes flow through the same create: regular path; that is reconfiguration, and it is what the guard is for, but it is not a rename or a rebuild.

7. The Delivery Gate: 07_delivery_check.yml

Every phase in this series ends by proving what it built. Phase 4 automates that instinct into the pipeline itself: the last play of every deploy is a checklist whose green output is the handover record. A VM that fails the gate is not delivered, even if every earlier play was green; that is the entire point of putting the gate last.

Check	How it is proven	Why this and not something weaker
Services active	service_facts plus asserts on zabbix-agent2, rsyslog, qemu-guest-agent	state: running in a play is a request; service_facts is an observation
Mounts up	ansible_facts.mounts must contain every data disk's mount point	a built filesystem that is not mounted is a DB server waiting to fail
Agent markers	every security_agents entry's .installed file must exist	06's marker contract; the marker is the truth, not the installer output
Forwarding established	ss -tn state established against the receiver port, with retries	the disk queue buffers when the link is down; delivery means the link is up
Tagged message	logger with tag faz4-<hostname>, found in the local log	proves rsyslog actually processed a message, not just that it is running
Zabbix listening	ss -tln on port 10050	active checks need the listener; server-side registration is a Zabbix-side action

Table 7. The six checks of the delivery gate.

The gate reads the same group_vars sections the plays above it wrote against, so its expectations cannot drift from its build. A failure stops the pipeline loudly, with a fail_msg that names the diagnostic command for the failing check; the series' position on quiet warnings is unchanged since Phase 2. What the gate deliberately does not check is anything outside the VM: the Zabbix server's knowledge of the host, the QRadar-side ingestion, the licensed agents' real health. Those are other systems' jobs, and pretending to verify them from the guest would be the dishonest kind of green.

8. Troubleshooting

The matrix follows the series convention: the error as the field will see it, the cause, and the fix. The first eight rows are Phase 4's own discoveries: two verified in the sandbox before they could become someone's evening, four found the hard way on sem-01 during the first field install, and two more found when the guide's own chapter 4 ran for real on ctrl-01 - the errata fixed in this edition, from the master/main default to a placeholder the PDF renderer silently ate. Six field findings from one install is a lesson about testing the whole path, not the happy pieces: the sandbox exercised the binary directly, as its own user, against the front page, and never crossed the boundaries the systemd unit, the inherited environment, the login endpoint, and the printed page itself cross on every real run. The remaining rows are the bugs the logic tests caught, for the same reason: "the tests caught it" is only reassuring if the bug is documented afterwards.

Symptom	Cause	Fix
git push answers "error: src refspec main does not match any" at the chapter 4 step (field-found on ctrl-01, 28 Aug 2026; guide errata)	plain git init still creates branch master - the default changes only in Git 3.0 - while the bare repo, the push command, and the Semaphore repository object all say main, so the push names a branch that does not exist; the original chapter 4 block ran git init without -b main	on the already-committed copy: git branch -m master main, then push. The corrected chapter 4 block uses git init -b main from the start
authorized_keys ends up empty - a later clone from sem-01 answers "Permission denied (publickey)"; copy-pasting the original block also produced "chmod: missing operand" (field-found on ctrl-01, 28 Aug 2026; guide errata)	two errata in the original chapter 4 block: the placeholder <sem-01 deploy public key> was silently eaten by the PDF renderer (the guide library's esc() escaped nothing, and the paragraph parser treats <word> as markup), and the 89-character chown+chmod line wrapped across two PDF lines, so the pasted chmod ran without its operand	this edition writes the lab's real deploy key into the block, keeps every line short enough not to wrap, and esc() escapes for real; ad-hoc on an existing box: echo "<deploy key>" > /home/git/.ssh/authorized_keys, then chown and chmod as separate lines
sem-01-setup.sh stops before step 1/8, after "tr: write error: Broken pipe"	head -c 16 closes the pipe after 16 bytes; tr dies on the write error; under set -euo pipefail the status rides the command substitution into the password assignment and aborts the script (field-found on sem-01, 28 Aug 2026; the sandbox run missed it because it passed the password as an argument)	fixed in the script: head reads a bounded 256-byte block first and cut trims to 16, so every exit status stays zero. Workaround on an already-pushed copy: run the script with the password as its first argument
ansible-galaxy inside sem-01 dies with "Ansible could not initialize the preferred locale: unsupported locale setting"	pct exec inherits the calling shell's LANG (en_US.UTF-8 on pve-a), which the minimal container has not generated; ansible-core refuses an invalid locale. The perl locale warnings in the same output are the cosmetic cousin (field-found on sem-01, 28 Aug 2026)	fixed in the script: LANG/LC_ALL exported to C.UTF-8 (built into glibc), the locales package installed, en_US.UTF-8 generated so future inherited sessions are valid too. Ad-hoc form: env LANG=C.UTF-8 in front of the command

Symptom	Cause	Fix
journalctl shows "Cannot Find configuration! Use --config parameter to point to a JSON or YAML file"; the unit loops on activating (auto-restart), exit status 1 (restart counter 445 after one hour)	the message is the binary's catch-all for an UNREADABLE config, not only a missing one: root-created state meets a non-root unit. The migrations and the admin creation ran as root, so the SQLite database is root-owned, and config.json is 640 root:root; the User=semaphore unit can read neither the config nor the database (field-found on sem-01, 28 Aug 2026)	fixed in the script: the config is made group-readable by the service user and the data directory is handed over recursively after the migrations. Ad-hoc: chown root:semaphore /etc/semaphore/config.json; chown -R semaphore:semaphore /var/lib/semaphore; systemctl restart semaphore
First sign-in shows "Failed to create session"; POST /api/auth/login answers HTTP 500 although the password is correct	v2.19 signs session cookies and requires two secrets in config.json: cookie_hash and cookie_encryption. Without them the server starts, the form renders, bcrypt passes, and the cookie encoding fails. The older field names cookie_hash_key / cookie_encryption_key still circulating in guides are silently ignored; `semaphore setup` generates the keys, a hand-written minimal config does not (field-found on sem-01, 28 Aug 2026, and reproduced in the sandbox: same binary, same flags, POST to the login endpoint)	fixed in the script: both keys generated with pipefail-safe od -N and written into the config. Ad-hoc: generate a 64-hex-char and a 32-hex-char value, add cookie_hash and cookie_encryption to config.json, restart the service. Verified round trip in the sandbox: login 204 with Set-Cookie, authenticated GET /api/user 200
json: cannot unmarshal number into Go struct field ConfigType.port of type string	config.json has "port": 3000 (a number); the binary requires a string	quote it: "port": "3000". sem-01-setup.sh writes it correctly
Semaphore .deb installed but no service exists	the .deb ships only /usr/bin/semaphore, no unit, no config	the setup script creates /etc/semaphore/config.json and the systemd unit from the official template
Clone succeeded but the VM has the template's CPU, RAM, and network	proxmox_kvm's clone branch forwards only format, full, pool, snapname, storage, target; spec parameters are dropped	the two-task pattern: clone first, then update: true (with update_unsafe: true for net and ide); never rely on specs riding the clone call
Re-ran the deploy; a second VM with the same name appeared	clone mode allocates the next free VMID without checking names	the playbook's proxmox_vm_info guard prevents this; if you bypass it, remove the duplicate VM and re-run

Symptom	Cause	Fix
pip install fails with externally-managed-environment on sem-01	PEP 668 on Ubuntu 26.04; the Phase 1 finding, same wall	the setup script uses apt packages (ansible, python3-proxmoxer, python3-requests); a venv under the service user is the fallback
Syntax error in template: expected token ',', got 'for'	a Jinja dict literal inside a list comprehension breaks the parser	the derivation uses a loop with list append, the same pattern 02_storage.yml always used
'final_data_disks' is undefined on a diskless profile	the derivation loop skipped itself on an empty disk list, leaving the fact unset	the playbook initializes the list to empty before the loop; caught by the web-small logic test
Serial not visible in the guest after attaching a disk to a RUNNING VM	the Phase 2 field finding: a new disk's serial may reach the guest only at the next full VM start	qm shutdown <vmid> && qm start <vmid> on the PVE node; a guest reboot is not enough. The Phase 4 path attaches disks before first start, so first deploys never hit this
group_vars not loading for the dynamically added host	the provisioned group must exist in the inventory file for group_vars/provisioned.yml to attach to add_host members	provisioning.yml declares the empty group on purpose; do not trim it

Table 8. The Phase 4 troubleshooting matrix.

9. What Phase 5 Holds

Phase 4 closes the delivery loop, and it closes it with the scope deliberately tight: no TLS, no high availability, no load testing, no customer self-service. The backlog is not empty; it is ordered.

- TLS on the log path (port 6514): the forwarding rule, the certificates, the QRadar-side ingestion. The rsyslog role's variables already isolate the protocol decision.
- The resize button: a Semaphore template wrapping proxmox_disk state: resized plus a re-run of 02_storage.yml, the two-command chain from Phase 2 wearing a form.
- Load testing: the honest EPS limit of the forwarding path, queue.maxdiskspace under disk pressure, and the numbers the SIEM team will ask for.
- HA and runners: a second Semaphore instance or remote runners against PostgreSQL, the env-var switch documented in chapter 3.
- The real agents: swapping the stub installers for licensed binaries under the same marker contract, the swap-in runbook from the Phase 3 guide.

- The AWX watch: if a 25.x release revives the project, the engine is UI-agnostic by design and the migration is a wiring exercise, not a rewrite.

The series convention also demands a runbook edition 2: the daemon behavior corrections from the Phase 3 field runs (the DA queue file that neither shrank nor disappeared on drain), the dpkg provenance closure, and now the Phase 4 module behaviors and the port-as-string finding. Edition 2 collects the corrections in one place with the field receipts attached; it is written when Phase 5 starts, against the same rule as everything else in this series: claims get verified, errors get kept, and the receipts get filed.